

TALLINNA TEHNIKAÜLIKOOL
Infotehnoloogia teaduskond
Arvutisüsteemide instituut

Frank Christopher Kirch 221925IVCM

KODUTÖÖ 1: FUNKTSIOONI $Y = F(X)$ TABULEERIMINE

Kodutöö

Juhendaja: Risto Heinsar
Insener

Tallinn 2023

Autorideklaratsioon

Kinnitan, et olen koostanud antud kodutöö iseseisvalt ning seda ei ole kellegi teise poolt varem kaitsmisele esitatud. Kõik töö koostamisel kasutatud teiste autorite tööd, olulised seisukohad, kirjandusallikatest ja mujalt pärinevad andmed on töös viidatud.

Autor: Frank Christopher Kirch

29.11.2023

Lühendite ja mõistete sõnastik

N	Sammude arv
H	Sammude pikkus
A	Funktsiooni argumendi algväärtus
B	Funktsiooni argumendi lõppväärtus
UML	Unified Modeling Language

Sisukord

Autorideklaratsioon.....	2
Lühendite ja mõistete sõnastik.....	3
Sisukord.....	4
Jooniste loetelu.....	6
Tabelite loetelu.....	7
1 Ülesande püstitus.....	8
1.1 Ülesande variant.....	8
Funktsiooni analüüs.....	9
Funktsiooni graafik.....	9
2 Lahenduse kirjeldus.....	11
2.1 Programmi töövoog.....	11
2.2 Funktsioonid.....	12
2.2.1 Põhiprogramm <i>main()</i>	12
2.2.2 Põhiülesande põhifunktsioon <i>BaseSolution</i>	13
2.2.3 Põhiülesande funktsioon <i>PrintGreetMessage</i>	13
2.2.4 Põhiülesande funktsioon <i>GetStartInput</i>	14
2.2.5 Põhiülesande funktsioon <i>GetEndInput</i>	14
2.2.6 Põhiülesande funktsioon <i>GetStepInt</i>	15
2.2.7 Põhiülesande funktsioon <i>ModifyStepIntoBound</i>	16
2.2.8 Põhiülesande funktsioon <i>CalcH</i>	16
2.2.9 Põhiülesande funktsioon <i>GetArrayX</i>	17
2.2.10 Põhiülesande funktsioon <i>GetArrayY</i>	18
2.2.11 Põhiülesande funktsioon <i>PrintTable</i>	19
2.3 Algoritm.....	19
3 Lisaülesanne.....	22
3.1 Eriolukordade analüüs.....	24

4 Kokkuvõte.....	26
Lisa 1 – Kuvatõmmised.....	27

Jooniste loetelu

Joonis 1: Funktsiooni graafik nullpunkti ümbruses.....	9
Joonis 2: Funktsiooni graafik positiivse argumenti x korral.....	10
Joonis 3: Funktsiooni graafik negatiivse argumenti x korral.....	10
Joonis 4: UML tegevusdiagramm baasüleande lahenduse korral.....	20
Joonis 5: UML tegevusdiagramm programmi režiimi valimisel.....	22
Joonis 6: Lisasülesande põhiprogrami osa.....	23
Joonis 7: Lisasülesande abifunktsioon TestHardCodedValues.....	23

Tabelite loetelu

Tabel 1. Eriolukordade analüüs.....	24
-------------------------------------	----

1 Ülesande püstitus

Kodutöö 1 kätkeb funktsiooni lahendamist ning kuvada tulemused tabelisse. Ülesande lahendusega täitub tabel funktsiooni argumendi x ja y väärtused, lähtuvalt sisendandmetest. Funktsiooni analüüsimisel on lähtutud funktsiooni graafikust.

Põhiülesande algoritm on põhjalikult kirjeldatud UML tegevusdiagrammiga [Joonis 4]. Töö sisaldab ka lisaülesannet, milles on lahendatud kolme komplekti eelseadistatud sisendandmeid. Lisaülesande algoritm kasutab põhiülesandes kirjeldatud UML tegevusdiagrammi [Joonis 4], ent rakendab ka lisaülesande käivitamist selgitavat algoritmi, mis on kirjeldatud eraldiseisva UML tegevusdiagrammiga [Joonis 6].

Lühidalt, käesoleva ülesande lahenduse tulemusel leiab kasutaja võrdsete vahedega funktsiooni abstsissid ning vastavad ordinaadid, võttes arvesse kasutaja enda poolt soovitud abstsisside sammude arvu, ning arvestatades ka sisestatud argumenxi x alg- ja lõppväärtust.

1.1 Ülesande variant

Kodutöö lahendamisel genereeriti variant lähtuvalt tudengi matriklinumbrit. Selle töö varandi genereerimiseks kasutati matriklinumbrit 221925.

Selle variandiga genereeri, et sisendiks on argumendi x alg- ja lõppväärtused A ja B ning sammude arv N .

Kehtivad tingimused $A < B$; $N > 0$.

Samm $H = (B - A) / (N - 1)$. Funktsiooni väärtust y arvutatakse punktides:

A

$A + H$

$A + 2H$

...

$A + (N-1)H$.

Funktsioon

$$y = (2x+1)/\sqrt{(2x^2+3x+4)^3-7}$$

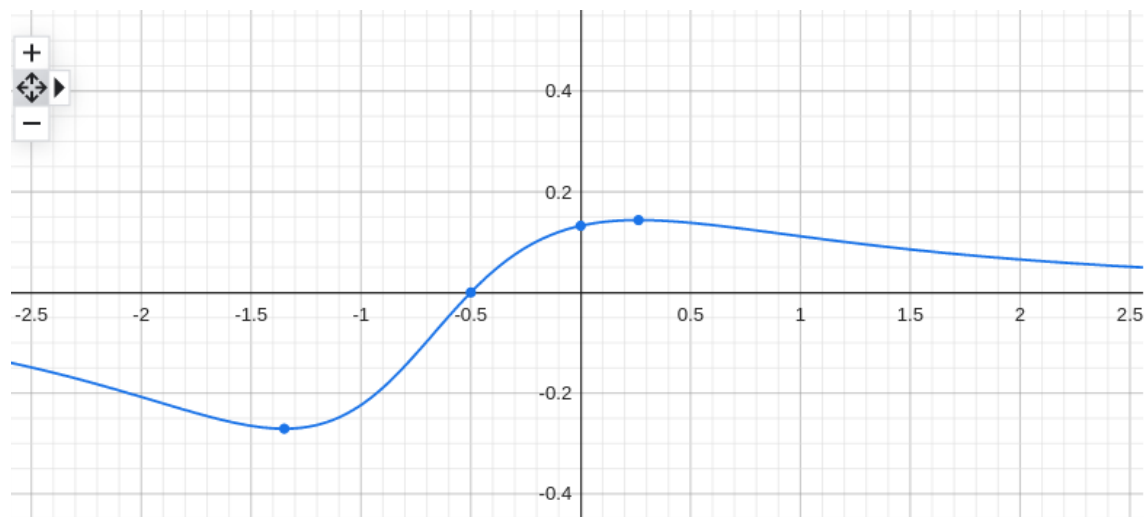
Funktsiooni $y=f(x)$ analüüs

Funktsiooni analüüs võimaldab tuvastada järgmiseid tunnuseid:

- Funktsiooni määramispiirkond: $x = [-\infty; \infty]$
- Funktsiooni muutumispiirkond: $y = [-0.271, 0.144]$
- Funktsiooni lõikepunkt x-teljega on $(-0.5, 0)$.
- Funktsiooni lõikepunkt y-teljega on $(0, 0.132)$.
- $Y_{\min} = (1.349; -0.271)$; $Y_{\max} = (0.263, 0.144)$;

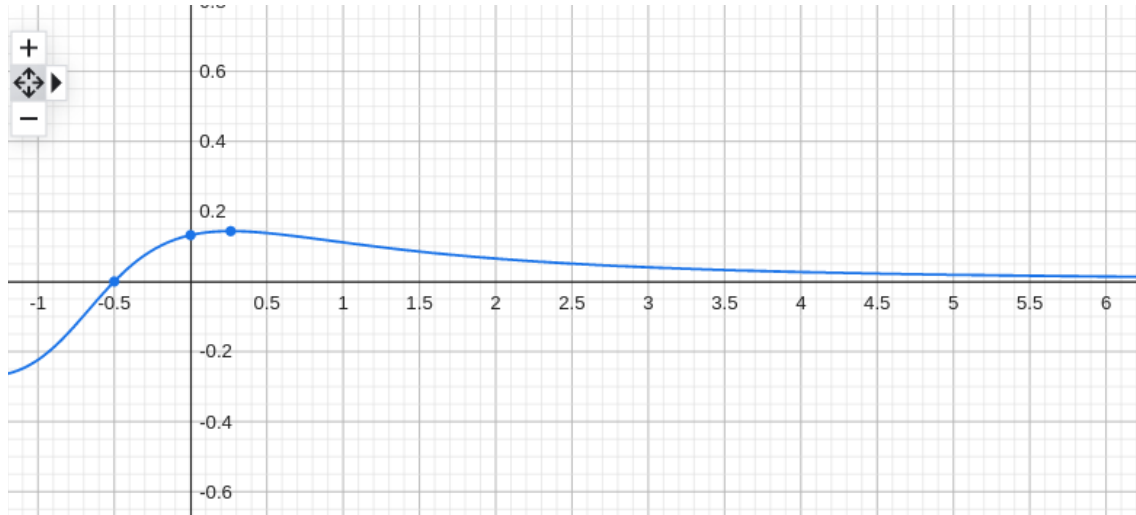
Funktsiooni $y=f(x)$ graafik

Funktsiooni graafik on ainulaadne, ning selget erineb tavapärastest ruutvõrranditest. Funktsiooni nimetaja ja lugeja koosmõjul moodustub funktsioon.



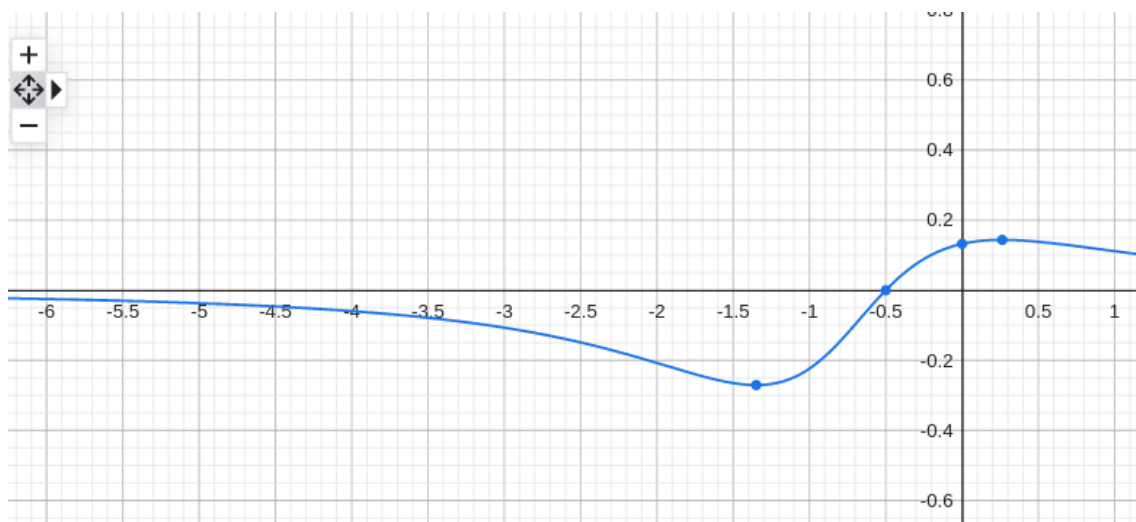
Joonis 1: Funktsiooni graafik nullpunkti ümbruses.

Funktsiooni x -teljest paremale poole on näha, et mida suurem on abstsiss (x), seda lähedamal on ordinaadi (y) väärtus arvule 0. See on nähtav alltoodud funktsiooni graafiku osast:



Joonis 2: Funktsiooni graafik positiivse argumendi x korral.

Jällegi, toimub sarnane muster x -teljest vasakul pool – mida väiksem on abstsiss (x), seda lähedamal on ordinaadi (y) väärtus arvule 0.



Joonis 3: Funktsiooni graafik negatiivse argumendi x korral.

Funktsiooni graafiku joonised 1-3 on näha funktsiooni määramis- ja muutumiskiirkonda, lõikepunkte x - ja y -telgedega ning absoluute ($Y_{\min}$ ja $Y_{\max}$).

2 Lahenduse kirjeldus

Programmi eesmärk on leida funktsiooni abstsisse ning ordinaate, küsides kasutajalt mitut tulemust soovib ta näha, mida käsitletakse ülesandes niinimetatud sammude arvuna. Programm küsib ka esimest abstsissi ja viimast abstsissi, ning mis kõige olulisem, programm arvutab nende kolme sisendi põhjal sammi pikkuse. Sammu pikkus on teisisõnu kui mitu punkti tuleb liikuda järgmise abstsissi leidmiseks. Leitud abstsissi põhjal leitakse funktsiooni ordinaat. Programm sooritab neid arvutustehti võrdselt sammude arvuga ning kuvab leitud vastuse kasutajale käsureal.

Programm ootab kolme sisendit: funktsiooni argumendi x alg- ja lõppväärtust, ning sammude arvu nende vahel. Viimane neist on täisarv, ning ülejäänud aga andmetüüp nimega `double`. Programm väljastab oodatavad tulemused komakohti lubava andmetüübiga `double`. Programm prindib ka vastavalt vajadusele konsooli veateateid, mis on andmetüübilt massiivid üksikutest tähtedest (ing. k. *character array*). Programm toimib abifunktsioonide väljakutsete toel.

2.1 Programmi töövoog

1. Programm informeerib kasutajat programmi eesmärgist ning kuvab võrrandit.
2. Programm küsib kasutajalt sisendandmeid kuniks sobituvad ülesande tingimustega.
3. Programm arvutab lähtuvalt sisendandmetest sammu pikkuse tähisega H .
4. Programm arvutab funktsiooni abstsissi (x) lähtuvalt sisendandmetest ja sammu pikkusest (H), ning talletab leitu abstsisside massiivi.
5. Programm arvutab funktsiooni ordinaadi y lähtuvalt sisendandmetest ja abstsissist (x), ning talletab leitud ordinaadite massiivi.
6. Programm kuvab funktsiooni abstsissid (x -d) ja ordinaadid (y -d) ning struktureerib tabelisse.

2.2 Funktsioonid

2.2.1 Põhiprogramm *main()*

Ülesande põhifunktsioon *main()* võtab sisendiks järgmised argumendid:

1. käsurea argumendite arvu, mis on täisarvu andmetüübiga (ing. k. *integer*)
2. käsurea argumendi väärtused, mis on tähestiku sümbolitest koosnev massiiv (ing. k. *Character array*)

```
68  /** Defining main body of the program, which is called out by Running the code. */
69  int main(int argc, char *argv[])
70  {
71      /** Creating variables in main function. Global variables forbidden! */
72      int exitCode = 1;
73
74      /** Checking if user suitable passed command-line argument */
75      if (argc == 2)
76      {
77          if (strcmp(argv[1], "--TEST1") == 0)
78          {
79              TestHardcodedValues(TEST_1_MIN, TEST_1_MAX, TEST_1_STEP, MAX_STEP_NR, PRESET_TEST_1);
80              return 0;
81          }
82          else if (strcmp(argv[1], "--TEST2") == 0)
83          {
84              TestHardcodedValues(TEST_2_MIN, TEST_2_MAX, TEST_2_STEP, MAX_STEP_NR, PRESET_TEST_2);
85              return 0;
86          }
87          else if (strcmp(argv[1], "--TEST3") == 0)
88          {
89              TestHardcodedValues(TEST_3_MIN, TEST_3_MAX, TEST_3_STEP, MAX_STEP_NR, PRESET_TEST_3);
90              return 0;
91          }
92          else
93          {
94              printf("\nWrong input\n");
95              return 1; /** exiting program (incorrect terminal argument) */
96          }
97      }
98      /** User did not pass command-line argument. Thus user can give inputs. */
99      else
100      {
101          do
102          {
103              BaseSolution(); /**run program 1st time regardless (do-while-loop) */
104              /** Re-looping entire program until user input's 0 at terminal prompt.*/
105              printf("\nPlease enter '0' to exit the program or anything other number to repeat the prog
106              scanf("%d", &exitCode);
107          }
108          while (exitCode != 0);
109      }
110      return 0; /** exiting program (program ran) */
111  }
112 }
```

Lisa 1: Funktsioon *main()*

Funktsioon *main()* võimaldab kasutajal valida, kas lahendab põhi- või lisäülesannet.

2.2.2 Põhiülesande põhifunktsioon *BaseSolution*

Funktsioon lahendab põhiülesande, initialiseerides muutujad ja kasutades neid abistavates funktsioonides. Funktsioonil pole argumente ega tagasta väärtust.

```
176 /**  
177  * Description:    Call functions to ask inputs, solve 'y = Fn(x)' and print.  
178  *   
179  * Parameters:    -  
180  * Return:        -  
181  */  
182 void BaseSolution()  
183 {  
184     double startInput = 0.00;  
185     double endInput = 0.00;  
186     int stepInput = 0;  
187  
188     double valueH = 0;  
189  
190     double arrayX[MAX_STEP_NR] = {0};  
191     double arrayY[MAX_STEP_NR] = {0};  
192     int xArrayLenCnt, yArrayLenCnt;  
193  
194     PrintGreetMessage();  
195  
196     /** A. Asking Function's argument starting value (X0 = A)**/  
197     startInput = GetStartInput();  
198  
199     /** B. Asking Function's stopping value (B) **/  
200     endInput = GetEndInput(startInput);  
201  
202     /** C. Asking Function's step (N) - (i.e how many x's user wants?). **/  
203     stepInput = GetStepInt(MAX_STEP_NR);  
204  
205     /** D. Re-adjusting step (N) to '20' (MAX_STEP_NR), if is larger than 20 (MAX_STEP_NR). **/  
206     int newStepInput = ModifyStepIntoBound(stepInput, MAX_STEP_NR);  
207  
208     /** E. Calculate step size (H). (i.e how large is the difference between x1 - x2) **/  
209     valueH = CalcH(startInput, endInput, newStepInput);  
210  
211     /** G. Fill arrayX and return the length of the arrayX. --> is this VLA?**/  
212     xArrayLenCnt = GetArrayX(arrayX, startInput, newStepInput, valueH);  
213  
214     /** G. Fill arrayY and return the length of the arrayY. --> is this VLA?**/  
215     yArrayLenCnt = GetArrayY(arrayY, arrayX, xArrayLenCnt);  
216  
217     /** H. Print results into a table, acquiring values from two arrays (x and y)**/  
218     PrintTable(arrayX, arrayY, yArrayLenCnt, stepInput);  
219  
220 }
```

Lisa 2: Funktsioon *BaseSolution()*

2.2.3 Põhiülesande funktsion *PrintGreetMessage*

Funktsioon *PrintGreetMessage()* ainus ülesanne on tervitada kasutajat programmi käivitamisel. Funktsioon ei tagasta, vaid prindib konsooli. Funktsioonil ei ole ka parameetreid.

```

114  /**
115  * Description:   Display introductory information to user of
116  *               the Function:  $y = (2x+1) / \sqrt{(2x^2 + 3x + 4)^3 - 7}$ 
117  * Parameters:   -
118  * Return:       -
119  */
120  void PrintGreetMessage()
121  {
122      printf("\nThe program solves function  $y = (2x+1) / \sqrt{(2x^2 + 3x + 4)^3 - 7}$ \n");
123      printf("The function's domain is:  $[-\infty; \infty]$ .\n");
124  }

```

Lisa 3: Funktsioon *PrintGreetMessage()*

2.2.4 Põhiülesande funktsioon *GetStartInput*

```

221  /**
222  * Description:   Ask user input about start of the function point.
223  *
224  * Parameters:   -
225  * Return:       double: starting x of the function, by user input.
226  */
227  double GetStartInput()
228  {
229      double startInput;
230      printf("\nPlease enter Starting point of Function: ");
231      scanf("%lf", &startInput);
232      return startInput;
233  }

```

Lisa 4: Funktsioon *GetStartInput*

Funktsioon *GetStartInput* küsib kasutajalt argumendi x algväärtust, ning tagastab selle initsialiseeritud *double* andmetüübina.

2.2.5 Põhiülesande funktsioon *GetEndInput*

Funktsioon *GetEndInput* küsib kasutajalt funktsiooni argumendi x lõppväärtust senikaua, kui kasutaja sisestab väärtuse, mis on suurem kui funktsiooni *GetStartInput* tagastatav väärtus [Lisa 4]. Funktsioon tagastab väärtuse *double* andmetüübina.

```

234 /**
235  * Description:   Ask user input about end of the function point.
236  *
237  * Parameters:   -
238  * Return:       double: Last/ending x of fuction by user input.
239  */
240 double GetEndInput(double startInput)
241 {
242     double endInput;
243     while(1)
244     {
245         printf("\nPlease enter Ending point of Function: ");
246         scanf("%lf", &endInput);
247         if (endInput <= startInput)
248         {
249             printf("Try again! End value was %6.3lf. It cannot be smaller than start value %6.3lf",
250                 endInput, startInput);
251         }
252         else
253         {
254             break;
255         }
256     }
257     return endInput;
258 }

```

Lisa 5: Funktsioon *GetEndInput*

2.2.6 Põhiülesande funktsioon *GetStepInt*

Funktsioon *GetStepInt* küsib kasutajalt sammude arvu senikaua, kui see on suurem kui

1. Funktsioon tagastab väärtuse täisarvuna (ing. k. *integer*) andmetüübina.

```

260 /**
261  * Description:   Ask user input step of function points
262  *
263  * Parameters:   integer: largest allowed step ('20') (MAX_STEP_NR)
264  *
265  * Return:       integer: user inputted step that is a whole number and not larger than 20.
266  */
267 int GetStepInt()
268 {
269     int stepInput = 0;
270
271     while(1)
272     {
273         printf("\nPlease enter Step of Function: ");
274         scanf("%d", &stepInput);
275         if (stepInput <= MIN_STEP)
276         {
277             printf("Please try again! Step must be more than 0. (chosen input: '%d')\n", stepInput);
278         }
279         else if (stepInput == UNDIVISABLE_STEP)
280         {
281             printf("Please try again! ");
282             printf("Step value cannot be '%d', because equation is then unsolvable: ", stepInput);
283             printf("'H = (B - A) / (N - 1)'\n");
284         }
285         else
286         {
287             break;
288         }
289     }
290     return stepInput;
291 }
292

```

2.2.7 Põhiülesande funktsioon *ModifyStepIntoBound*

Funktsioon *ModifyStepIntoBound* kirjutab kasutaja sisestatud sammude arvu üle uue väärtusega, mis on võrdne maksimaalselt lubatud sammude arvuga, juhul kui kasutaja on sisestatud sammude arvuks rohkem kui seda on lubatud. Funktsioon tagastab täisarvu (ing. k. *integer*).

```
294  /**
295   * Description:      Re-adjusting too large number of steps (N) into maximum allowed step nr.
296   *
297   * Parameters:      number of steps (N), maximum allowed number of steps (N)
298   *
299   * Return:          int: number of steps (N) (adjusted)
300   */
301  int ModifyStepIntoBound(int stepValue, int maxStep)
302  {
303      stepValue = (stepValue > maxStep) ? maxStep : stepValue;
304      return stepValue;
305  }
```

Lisa 6: Funktsioon *ModifyStepIntoBound*

2.2.8 Põhiülesande funktsioon *CalcH*

Funktsioon *CalcH* võtab argumentideks algväärtuse, lõppväärtuse ja sammude arvu, ning tagastab ühe sammu pikkuse *double* andmetüübina.

```
307  /**
308   * Description:      Calculate H --> H = (B - A) / (N - 1). Re-asking if undivisable. --->
309   *
310   * Parameters:
311   *      A = starting x value,
312   *      B - stopping x value,
313   *      N = steps in function argument
314   *      maximum allowed nr of steps. (MAX_STEP_NR)
315   *
316   * Return:          double: size of the step (i.e distance/difference between x0 and x1)
317   */
318  double CalcH(double startValue, double stopValue, int stepValue)
319  {
320      return (double)(stopValue - startValue) / (stepValue - 1); /** return valueH; */
321  }
```

Lisa 7: Funktsioon *CalcH*

2.2.9 Põhiülesande funktsioon *GetArrayX*

Funktsioon *GetArrayX* võtab argumendiks argumendi *x* alg- ja lõppväärtuse, sammu pikkuse (*H*), ning initialiseeritud massiivi leitavate funktsiooni *x* argumentide hoiustamiseks. Funktsioon leiab abstsissid ja lisab need massiivi. Funktsioon tagastab arvu abstsisse, mis hoiustatakse massiivis, teisisõnu massiivi pikkuse.

```
333 int GetArrayX(double xArray[], double startValue, int stepValue, double valueH)
334 {
335     int xArrLenCnt = 0;
336     double valueX = 0.00;
337     int i = 0;
338
339     for (i = 0; i < stepValue; i++)
340     {
341         valueX = (double)(startValue + (xArrLenCnt * valueH)); /** converting i
342         xArray[xArrLenCnt] = valueX; /** storing x (smaller-equal of stopVal
343         xArrLenCnt++; /** incrementing count. */
344     }
345     /** in this function I place valueX per iteration into xArray until limit (
346     return xArrLenCnt-1;
```

Lisa 8: Funktsioon *GetArrayX*

2.2.10 Põhiülesande funktsioon *GetArrayY*

Funktsioon *GetArrayX* võtab argumendiks massiivi abstsissidest ja selle massiivi pikkuse, ning initialiseeritud massiivi leitavate funktsiooni *y* argumentide hoiustamiseks. Funktsioon leiab ordinaadid ja lisab ordinaatide massiivi. Funktsioon tagastab arvu ordinaate, mis hoiustatakse massiivis, teisisõnu massiivi pikkuse.

```
349  /**
350  * Description:   Populate array of Ys to the length of array X
351  *
352  * Parameters:   array of Ys, array of Xs, length of array X
353  *
354  * Return:      integer: length of array of Ys
355  */
356  int GetArrayY(double yArray[], double xArray[], int xArrLenCnt)
357  {
358      double dividend, divisor;
359      double valueX = 0.00;
360      double valueY = 0.00;
361
362      int yCnt = 0;
363      int i = 0;
364      for (i = 0; i <= xArrLenCnt; i++)
365      {
366          /** Taking valueX from array, calculating valueY based on valueX */
367          valueX = xArray[i];
368          dividend = (2 * valueX) + 1;
369          divisor = sqrt((pow((2 * pow(valueX, 2) + 3*valueX + 4)), (double)3)) - 7);
370          valueY = (dividend / divisor);
371
372          /** Populating yArray at index 'i' with calculated valueY */
373          yArray[yCnt] = valueY;
374          yCnt++;
375      }
376      return yCnt-1;
377  }
```

Lisa 9: Funktsioon *GetArrayY*

2.2.11 Põhiülesande funktsioon *PrintTable*

```
378 /**
379  * Description:   Print the Table in format with results.
380  *
381  * Parameters:   array of Xs, array of Ys, size of step (H),
382  *               length of array Y,
383  *
384  *               booleanTag (whether we have crossed Max allowed step nr)
385  *
386  * Return:       -
387  */
388 void PrintTable(double xArray[], double yArray[], int yArrLenCnt, int stepValue)
389 {
390     char *columnHeader[] = {"ind", "X", "Y" };
391
392     printf("\n-----");
393     printf("\n----- RESULTS -----");
394     printf("\n-----");
395     printf("\n-- | %3s | %7s | %7s | --", columnHeader[0], columnHeader[1], columnHeader[2]);
396     printf("\n-----");
397     int i = 0;
398     for (i = 0; i <= yArrLenCnt; i++) /** Modify?*/
399     {
400         /** Print results, and call array on specific position/index */
401         printf("\n-- | %3d | %7.3lf | %7.3lf | --", i+1, xArray[i], yArray[i]);
402     }
403     printf("\n-----\n");
404
405     /** Inform user that we limit the number of results displayed*/
406     if (i < stepValue)
407     {
408         printf("\nError! \nWe are not allowed to calculate more than %d steps of X! \nYour step input
409     }
410
411 }
```

Lisa 10: Funktsioon *PrintTable*

Funktsioon *PrintTable* võtab argumentidega massiivi abstsissidest, massiivi ordinaatidest ja sama ordinaatide massiivi pikkuse, ning sammude arvu. Funktsioon väljastab struktureeritud tabeli lahendustega, ning vajadusel väljastab kasutajale teate, et tema sisestatud liiga suur sammude arv vähendati enim lubatud sammude arvuni.

2.3 Algoritm

Ülesande lahendamiseks võeti kasutusele preprotsessoreid, muutujaid, funktsioone ning tingimuslauseid ja mitmeid tsükleid [Joonis 4]. Algoritmiks on UML tegevusdiagramm, mis on jaotatud neljaks niinimetatud vertikaalseks basseiniks [Joonis 4]. Selle meetodiga on esitatud UML tegevusdiagrammis segmenteeritud muutujate defineerimine tsüklitest ja tingimuslausetest. Koodis maagiliste arvu likvideerimiseks on toodud sisse hulga konstante ning defineeritud preprotsessoritena, mis on nähtavad ka joonisel vasakpoolseimast vertikaalsest basseinist [Joonis 4]. Koodi saab korrata

```

graph TD
    subgraph Initialization
        Start(( )) --> DefineConst[Define constant to find number results stored in array MAX_STEP=10]
        DefineConst --> DefineConst2[Define constants to avoid invalid user input about amount/steps of Xs then STEP=1]
        DefineConst2 --> InitVar[Initialize variable exitCode = 1]
        InitVar --> InitVar2[Initialize variable to store user's input of starting X (startUpst)]
        InitVar2 --> InitVar3[Initialize variable to store user's input of last X (endUpst)]
        InitVar3 --> InitVar4[Initialize variable to store user's input of 'amount of steps' (stepUpst)]
        InitVar4 --> InitVar5[Initialize array of Xs with a max length array(MAX_STEP, 0)]
        InitVar5 --> InitVar6[Initialize array of Ys with a max length array(MAX_STEP, 0)]
        InitVar6 --> InitVar7[Initialize variable to count Xs stored in the array of Xs (xLenArrayCntX)]
        InitVar7 --> InitVar8[Initialize variable to count Ys stored in the array of Ys (yLenArrayCntY)]
    end

    subgraph UserInput
        PrintFunc[Print function formula and domain] --> AskStart[Ask user input of starting X (startUpst)]
        AskStart --> ValidateStart[validate value of start X with a while(1) loop]
        ValidateStart --> AskLast[Ask user input of last X (endUpst)]
        AskLast --> ValidateLast[Is user's input of last X (endUpst) smaller or equal to user's input of start of (startUpst)?]
        ValidateLast --> EndPrint[Print error message that last X cannot be smaller or equal to start of X]
        EndPrint --> AskStart
        ValidateLast --> Comment1[Comment: validate 'amount of steps' with while(1) loop]
        Comment1 --> AskSteps[Ask user input 'amount of steps' (stepUpst)]
        AskSteps --> ValidateSteps[Is user's input regarding 'amount of steps' valid (larger than 0)?]
        ValidateSteps --> MinStep[stepUpst <= MIN_STEP]
        MinStep --> AskSteps
        ValidateSteps --> Amount0[amount of steps is smaller than 0]
        Amount0 --> PrintError0[Print error message that input cannot be smaller than 0 (stepUpst)]
        PrintError0 --> AskSteps
        ValidateSteps --> Amount1[amount of steps is equal to 1]
        Amount1 --> PrintError1[Print error message that input cannot equal 1]
        PrintError1 --> AskSteps
        ValidateSteps --> Amount2[amount of steps larger 1]
        Amount2 --> PrintError2[Print error message that input cannot be larger than 1]
        PrintError2 --> AskSteps
        Amount2 --> Overwrite[Overwrite input with maximum allowed 'amount of steps' (stepValue = MAX_STEP - 1)]
        Overwrite --> AskSteps
        Amount2 --> Comment2[Is user's input regarding 'amount of steps' larger than maximum allowed 'amount of steps'?]
        Comment2 --> CalcStepValue[stepValue = MAX_STEP - 1]
        CalcStepValue --> CalcStepValue
    end

    subgraph MathematicalOperations
        CalcStepValue --> FindSize[Find 'size of a step', from one to the next X and store into variable (value)]
        FindSize --> InitVar9[Initialize variable to store X's value (xLenX)]
        InitVar9 --> ZeroInitX[Zero-initialize variable to count Xs stored in the array (xLenArrayCntX)]
        ZeroInitX --> InitVar10[initialize loop counter i = 0]
        InitVar10 --> Decision1{ }
        Decision1 --> CalcX[Calculate value of a X given number of the step (value)]
        CalcX --> StoreX[Store X to array consisting Xs at given array's index]
        StoreX --> IncIndexX[i = i + 1]
        IncIndexX --> DecCountX[xLenArrayCntX = xLenArrayCntX - 1]
        DecCountX --> ZeroInitY[Zero-initialize variable to store Ys to be calculated (value)]
        ZeroInitY --> InitVar11[Are there more Xs in array of Xs? xLenArrayCntX <= xLenArrayCntX?]
        InitVar11 --> Decision2{ }
        Decision2 --> CalcY[Function formula Y = (2x+1) / sqrt(12x^2 + 3x + 4)*3 - 7)]
        CalcY --> StoreY[Store Y in array consisting of Ys at given array's index]
        StoreY --> IncIndexY[i = i + 1]
        IncIndexY --> DecCountY[yLenArrayCntY = yLenArrayCntY - 1]
        DecCountY --> Decision3{ }
        Decision3 --> CalcX
        Decision3 --> CalcY
    end

    subgraph PrintingResults
        PrintHeader[Print table header with step, Xs, Ys and Ys] --> InitCounter[Initialize loop counter as 0]
        InitCounter --> Decision4{ }
        Decision4 --> PrintIndex[Print the index of X, which is the step number]
        PrintIndex --> PrintX[Print the value of X given the index and array of Xs]
        PrintX --> PrintY[Print the value of Y given the index and array of Ys]
        PrintY --> IncCounter[i = i + 1]
        IncCounter --> PrintHeader
        PrintHeader --> Decision5{ }
        Decision5 --> PrintError3[Print error message that amount of steps as input is decreased to the maximum allowed 'amount of steps']
        PrintError3 --> Decision5
        Decision5 --> AskContinue[Ask user to either exit the program or repeat it]
        AskContinue --> ExitCode[exitCode = 0]
        ExitCode --> EndProgram(( ))
        AskContinue --> Decision6{ }
        Decision6 --> EndProgram
        Decision6 --> PrintHeader
    end
  
```

Põhiülesande koodi lahendamine nõuab kasutajalt sisendandmete küsimist, ning neist lähtuvalt sammude pikkuse (H) ja funktsiooni abstsisside ja ordinatide arvutamist [Joonis 4].

```

176  /**
177  * Description:   Call functions to ask inputs, solve 'y = Fn(x)' and print.
178  *               This is program's 'main'-Fn body (without Extra-Task)
179  * Parameters:   -
180  * Return:       -
181  */
182  void BaseSolution()
183  {
184      double startInput = 0.00;
185      double endInput = 0.00;
186      int stepInput = 0;
187
188      double valueH = 0;
189
190      double arrayX[MAX_STEP_NR] = {0};
191      double arrayY[MAX_STEP_NR] = {0};
192      int xArrayLenCnt, yArrayLenCnt;
193
194      PrintGreetMessage();
195
196      /** A. Asking Function's argument starting value (X0 = A)**/
197      startInput = GetStartInput();
198
199      /** B. Asking Function's stopping value (B) **/
200      endInput = GetEndInput(startInput);
201
202      /** C. Asking Function's step (N) - (i.e how many x's user wants?). **/
203      stepInput = GetStepInt(MAX_STEP_NR);
204
205      /** D. Re-adjusting step (N) to '20' (MAX_STEP_NR), if is larger than 20 (MAX_STEP_NR). **/
206      int newStepInput = ModifyStepIntoBound(stepInput, MAX_STEP_NR);
207
208      /** E. Calculate step size (H). (i.e how large is the difference between x1 - x2) **/
209      valueH = CalcH(startInput, endInput, newStepInput);
210
211      /** G. Fill arrayX and return the length of the arrayX. --> is this VLA?*/
212      xArrayLenCnt = GetArrayX(arrayX, startInput, newStepInput, valueH);
213
214      /** G. Fill arrayY and return the length of the arrayY. --> is this VLA?*/
215      yArrayLenCnt = GetArrayY(arrayY, arrayX, xArrayLenCnt);
216
217      /** H. Print results into a table, acquiring values from two arrays (x and y)**/
218      PrintTable(arrayX, arrayY, yArrayLenCnt, stepInput);
219
220  }

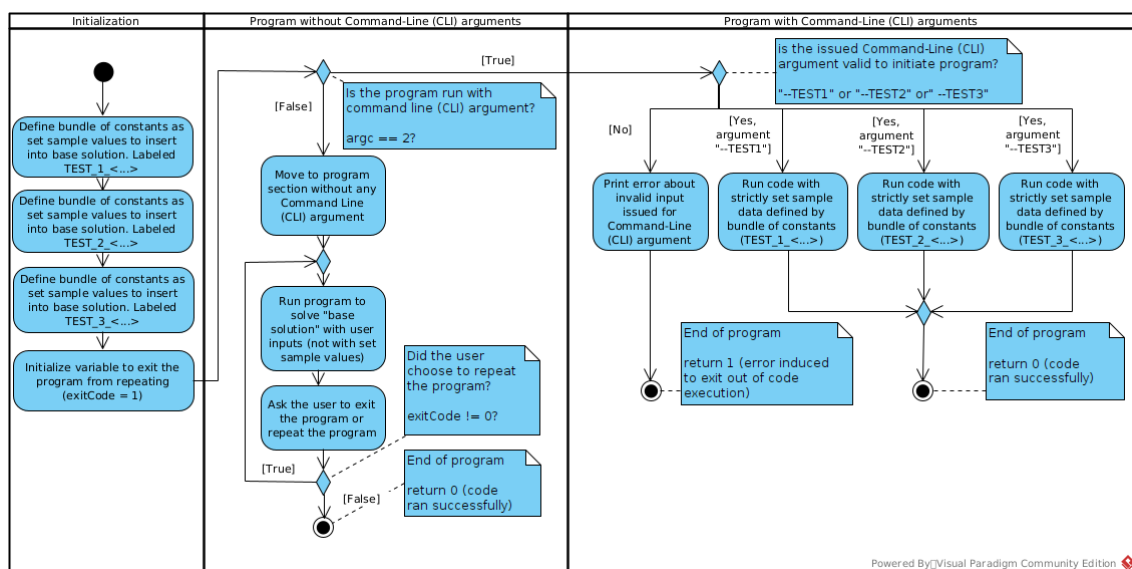
```

Lisa 11: Ülesande lahendus abifunktsiooni väljakutsetega

Ülaltoodud osa koodist lahendab põhiülesande, kutsudes välja abifunktsioone, mis on leitavad detailsemalt kodutöö koodifailist. Lisaülesanne lahendamiseks loodi funktsioone ning eeldefineeritud sisendandmed, mida kirjeldab järgmises peatükis.

3 Lisaülesanne

Lisaülesanne kirjeldus nõudis ülesande täiendamist vähemalt kahe eelseadistatud komplekti väärtusi, kus programmile antakse ette sisendandmed, mille põhjal leida funktsiooni abstiss x ja ordinant y igal otsitaval sammul. Lisaülesande korral tuleb kasutajal valida režiim, mis on algoritmina UML tegevusdiagrammil Joonis 4.



Joonis 5: UML tegevusdiagramm programmi režiimi valimisel

Koodi saab korrata senikaua kui kasutaja sisestab arvu 0, vastasel juhul kordab kood oma töövoogu algusest [Joonis 4]. Režiimist sõltumatult lahendab programm funktsiooni lähtuvalt algoritmile [Joonis 5].

Minu lahendus kätkeb kolme eelseadistatud komplekti väärtusi [Joonis 5], mis kutsutakse välja põhiprogrammis abifunktsiooniga TestHardcodedValues.

```

68  /** Defining main body of the program, which is called out by Running the code. */
69  int main(int argc, char *argv[])
70  {
71      /** Creating variables in main function. Global variables forbidden! */
72      int exitCode = 1;
73
74      /** Checking if user suitable passed command-line argument */
75      if (argc == 2)
76      {
77          if (strcmp(argv[1], "--TEST1") == 0)
78          {
79              TestHardcodedValues(TEST_1_MIN, TEST_1_MAX, TEST_1_STEP, MAX_STEP_NR, PRESET_TEST_1);
80              return 0;
81          }
82          else if (strcmp(argv[1], "--TEST2") == 0)
83          {
84              TestHardcodedValues(TEST_2_MIN, TEST_2_MAX, TEST_2_STEP, MAX_STEP_NR, PRESET_TEST_2);
85              return 0;
86          }
87          else if (strcmp(argv[1], "--TEST3") == 0)
88          {
89              TestHardcodedValues(TEST_3_MIN, TEST_3_MAX, TEST_3_STEP, MAX_STEP_NR, PRESET_TEST_3);
90              return 0;
91          }
92          else
93          {

```

Joonis 6: Lisasülesande põhiprogrami osa.

Koodilahenduses tuleb hoiduda maagilistest numbritest, mistõttu sisendandmed on defineeritud preprotsessoritena [Joonis 5], ning neid kasutatakse funktsiooni väljakutses [Joonis 6].

```

126  /**
127   * Description: Solving function with command line argument '--TEST2'
128   *              '--TEST2' CLI argument gives preset argument values.
129   *              Function  $y = (2x+1) / \sqrt{(2x^2 + 3x + 4)^3 - 7}$ 
130   *
131   *              NB! Function will not display all x (not all steps).
132   *              This is because 10th X-values is larger than stopping/ending x-value
133   * Parameters: -
134   *
135   * Return: -
136   */
137  void TestHardcodedValues(double startValue, double endValue, int stepValue, int maxStep, int testNr)
138  {
139      double valueH;
140      double arrayX[MAX_STEP_NR] = {0};
141      double arrayY[MAX_STEP_NR] = {0};
142      int xArrayLenCnt, yArrayLenCnt;
143
144      /** A. Printing greet-statements for user. */
145      PrintGreetMessage();
146      PrintPresetTable(startValue, endValue, stepValue, testNr);
147
148      int newStepValue = ModifyStepIntoBound(stepValue, maxStep);
149      valueH = CalcH(startValue, endValue, newStepValue);
150
151      xArrayLenCnt = GetArrayX(arrayX, startValue, newStepValue, valueH);
152      yArrayLenCnt = GetArrayY(arrayY, arrayX, xArrayLenCnt);
153
154      PrintTable(arrayX, arrayY, yArrayLenCnt, stepValue);
155  }

```

Joonis 7: Lisaülesande abifunktsioon TestHardCodedValues.

Ülaltoodud koodijupp [Joonis 7] rakendab funktsioonis TestHardcodedValues abifunktsioone, mis on osa baasülesande algoritmist [Joonis 4]. Kokkuvõtvalt, lisaülesanne hoidub kasutajalt sisendandmete küsimist, ning kasutab eeldefineeritud väärtusi ülesande lahendamiseks.

3.1 Eriolukordade analüüs

Tabel 1. Eriolukordade analüüs

#	Kirjeldus	Olek	Lahendus
1	Kasutaja sisestatud viimane funktsiooni abstsiss on väiksem või võrdne esimese funktsiooni abstsissiga	Lahendatud	Programm küsib kasutajalt uut väärtust viimaseks funktsiooni abstsissiks, ja senikaua kui see on suurem kui esimene abstsiss.
2	Kasutaja sisestab sammude arvuks (N) mittepositiivse arvu	Lahendatud	Programm küsib kasutajalt uut väärtust sammude arvuks, senikaua kui arv on suurem kui 0.
3	Kasutaja esitab sammude arvuks (N) rohkem kui lubatud arv samme	Lahendatud	Programm kirjutab üle kasutaja sisestatud sammude arvu väärtusega, mis on võrdne maksimaalselt lubatud sammude arvuga. Seejärel programm jätkub. Kasutajat teavitatakse sammude arvu (N) muutmisest.
4	Kasutaja sisestab ükskõik millisel juhul sisendandmeks arvu asemel tähemärgi	Lahendamata	Programm peaks kätkema endast andmetüübi valideerimist. Vastasel juhul programm sattub lõputusse tsükklisse, risustades konsooli.
5	Kasutaja sisestab sammude arvuks on komakohaga arvu	Lahendamata	Programm peaks kätkema endast andmetüübi valideerimist. Vastasel juhul programm sattub lõputusse tsükklisse, risustades konsooli.
Lisaülesanne			
6	Kasutaja sisestab ebakorrekse käsurea argumendi lisaülesande käivitamiseks	Lahendatud	Programm teavitab kasutajat vale käsurea argumendi esitamise pärast ning väljub programmist.
7	Lisaülesandes eelseadistatud sisendandmed on arvude asemel tähemärgid	Lahendamata	Programm peaks kätkema endast andmetüübi valideerimist. Vastasel juhul programm sattub lõputusse tsükklisse, risustades konsooli.
8	Eelseadistatud sammude arvuks	Lahendamata	Programm peaks kätkema endast andmetüübi valideerimist. Vastasel juhul programm sattub

	on komakohaga arvu		lõputusse tsüklisse, risustades konsooli.
9	Eelseadistatud viimane abstsiss on väiksem kui esimene abstsiss	Lahendamata	Programm jõuab lõpplahenduseni, kuid ei võta arvesse tingimust, et viimane abstsiss ei tohi olla väiksem kui esimene abstsiss.
10	Eelseadistatud sammude arv on suurem kui enim lubatud arv samme	Lahendatud	Programm kirjutab üle kasutaja sisestatud sammude arvu väärtusega, mis on võrdne maksimaalselt lubatud sammude arvuga. Seejärel programm jätkub. Kasutajat teavitatakse sammude arvu (N) muutmisest.

4 Kokkuvõte

Ülesande koodi lahendamiseks alustasin 3. oktoobril, ning töö kulges kuni 17. oktoobrini. Ülesande lahendasin mitme abi funktsiooniga, et lihtsustada koodiloetavust. Konsultatsioonides saadud tagasiside põhjal võtsin kasutusele **massiivid**, kuhu tõin funktsiooni abstsiss x ja ordinante y väärtused. Ülesanne oli lahendatav ka massiivideta, kuid ülesande kirjeldus nõudis massiivide kasutamist.

Ülesande keerukuseks hindan 9 punkti 10 punkti skaalal. Ülesande lahendamine nõudis õppeaines „IAX0583 Programmeerimine I” käsitletud varasemaid teemasid, mis oli ootuspärane. Keerukust lisas ülesande püstituses napsõnaliselt kirjeldatud muutujad tähisega H ja N . Ingliskeelses ülesande kirjelduses ei olnud mainitud, et H tähistab „sammu pikkust” (*ing. k size/length of a step*). Keeruliseks tegi ka ülesande puhul asjaolu, et ei olnud täpsustatud kas programmis järgmine juhtum – kui kasutaja sisestab rohkem kui lubatud arvu samme. Nõutav tingimus programmile oli mitmeti tõlgendatav, mistõttu oli raske mõista, kas programm peab vähendama kasutaja sisestatud sammude arvu numbrini, mis ühtib enim lubatud sammude arvuga. Näiteks, kui enim lubatud sammude arv on 20, ent kasutaja sisestab arvu 30. Kas seljuhul programm vähendab sammude arvu 20ni, ning lahendab funktsiooni, või programm peab küsima kasutajalt uue sobivama sammude arvu, mis oleks väiksem kui 20.

Ülesande lahendamisele kulus oodatust rohkem aega. Alustasin ülesande lahendamist 3. oktoobril, ning väga umbkaudselt kulus ülesande lahendamisele 40h-50h.

Soovitan edasiste ebaselguste vältimiseks ümbersõnastada ingliskeelses ülesandekirjelduses kaks mõistet. Minu ettepanek on kirjeldada kaks tähistatud muutujat, H ja N . Tähist H korral soovitan anda ingliskeelseks mõisteks „*size of a step*” või „*length of a step*”. Tähist N korral soovitan kasutada ingliskeelset vastet „*count of steps*” või „*number of steps*”. Kirjelduses oli mainitud vabatahtlik lisaülesanne imaginaararve leida, ning neid kuvada lõplikus lahenduses. Kahjuks imaginaararvu mõisteselgitus ja näited ülesande kirjeldusest puudusid. See raskendas imaginaararvude leidmise meetodi mõistmist.

Lisa 1 – Kuvatõmmised

```
frank@frankpc:~/Documents/C Programming/Homework01/Final$ ./HW1_221925IVCM_Frank_Christopher_Kirch

The program solves function  $y = (2x+1) / \sqrt{(2x^2 + 3x + 4)^3 - 7}$ 
The function's domain is:  $[-\infty; \infty]$ .

Please enter Starting point of Function: -2.225

Please enter Ending point of Function: 4.750

Please enter Step of Function: -1
Please try again! Step must be more than 0. (chosen input: '-1')

Please enter Step of Function: 0
Please try again! Step must be more than 0. (chosen input: '0')

Please enter Step of Function: 1
Please try again! Step value cannot be '1', because equation is then unsolvable: 'H = (B - A) / (N - 1)'

Please enter Step of Function: 5

-----
----- RESULTS -----
-----
-- | ind |      X |      Y | --
-----
-- |  1 | -2.225 | -0.179 | --
-- |  2 | -0.481 |  0.008 | --
-- |  3 |  1.262 |  0.097 | --
-- |  4 |  3.006 |  0.040 | --
-- |  5 |  4.750 |  0.021 | --
-----

Please enter '0' to exit the program or anything other number to repeat the program: 
```

Lisa 12: Põhiülesanne sisendväärtustega

```

frank@frankpc:~/Documents/C Programming/Homework01/Final$ ./HW1_221925IVCM_Fra
nk_Christopher_Kirch

The program solves function  $y = (2x+1) / \sqrt{(2x^2 + 3x + 4)^3 - 7}$ 
The function's domain is:  $[-\infty; \infty]$ .

Please enter Starting point of Function: -3.119

Please enter Ending point of Function: 6.234

Please enter Step of Function: -1
Please try again! Step must be more than 0. (chosen input: '-1')

Please enter Step of Function: 1
Please try again! Step value cannot be '1', because equation is then unsolvabl
e: 'H = (B - A) / (N - 1)'

Please enter Step of Function: 31

-----
----- RESULTS -----
-----
-- | ind |      X |      Y | --
-----
-- |  1 | -3.119 | -0.099 | --
-- |  2 | -2.627 | -0.137 | --
-- |  3 | -2.134 | -0.190 | --
-- |  4 | -1.642 | -0.252 | --
-- |  5 | -1.150 | -0.257 | --
-- |  6 | -0.658 | -0.076 | --
-- |  7 | -0.165 |  0.108 | --
-- |  8 |  0.327 |  0.143 | --
-- |  9 |  0.819 |  0.122 | --
-- | 10 |  1.311 |  0.095 | --
-- | 11 |  1.804 |  0.073 | --
-- | 12 |  2.296 |  0.056 | --
-- | 13 |  2.788 |  0.045 | --
-- | 14 |  3.280 |  0.036 | --
-- | 15 |  3.773 |  0.029 | --
-- | 16 |  4.265 |  0.025 | --
-- | 17 |  4.757 |  0.021 | --
-- | 18 |  5.249 |  0.018 | --
-- | 19 |  5.742 |  0.015 | --
-- | 20 |  6.234 |  0.013 | --
-----

Error!
We are not allowed to calculate more than 20 steps of X!
Your step input '31' is changed to maximum of '20'.

Please enter '0' to exit the program or anything other number to repeat the pr
ogram: 

```

Lisa 13: Põhiülesanne ebasobivate sisendväärtustega

```

frank@frankpc:~/Documents/C Programming/Homework01/Final$ ./HW1_221925IVCM_Fra
nk_Christopher_Kirch --TEST1

The program solves function  $y = (2x+1) / \sqrt{(2x^2 + 3x + 4)^3 - 7}$ 
The function's domain is:  $[-\infty; \infty]$ .

-----
-- Running TESTSET 1! --
-----
-- min    -5.555      --
-- max     5.554      --
-- step     30        --
-----

----- RESULTS -----
-----
-- | ind |      X |      Y | --
-----
-- |  1 | -5.555 | -0.029 | --
-- |  2 | -4.970 | -0.037 | --
-- |  3 | -4.386 | -0.049 | --
-- |  4 | -3.801 | -0.066 | --
-- |  5 | -3.216 | -0.093 | --
-- |  6 | -2.632 | -0.136 | --
-- |  7 | -2.047 | -0.201 | --
-- |  8 | -1.462 | -0.267 | --
-- |  9 | -0.878 | -0.180 | --
-- | 10 | -0.293 |  0.077 | --
-- | 11 |  0.292 |  0.144 | --
-- | 12 |  0.877 |  0.119 | --
-- | 13 |  1.461 |  0.087 | --
-- | 14 |  2.046 |  0.064 | --
-- | 15 |  2.631 |  0.048 | --
-- | 16 |  3.215 |  0.037 | --
-- | 17 |  3.800 |  0.029 | --
-- | 18 |  4.385 |  0.024 | --
-- | 19 |  4.969 |  0.019 | --
-- | 20 |  5.554 |  0.016 | --
-----

Error!
We are not allowed to calculate more than 20 steps of X!
Your step input '30' is changed to maximum of '20'.

```

Lisa 14: Lisaülesanne 1. eelseadistatud väärtuste komplektiga

```

frank@frankpc:~/Documents/C Programming/Homework01/Final$ ./HW1_221925IVCM_Fra
nk_Christopher_Kirch --TEST2

The program solves function  $y = (2x+1) / \sqrt{(2x^2 + 3x + 4)^3 - 7}$ 
The function's domain is:  $[-\infty; \infty]$ .

-----
-- Running TESTSET 2! --
-----
-- min    -4.500      --
-- max     5.550      --
-- step     11        --
-----

----- RESULTS -----
-----
-- | ind |      X |      Y | --
-----
-- |  1 |  -4.500 | -0.046 | --
-- |  2 |  -3.495 | -0.079 | --
-- |  3 |  -2.490 | -0.150 | --
-- |  4 |  -1.485 | -0.266 | --
-- |  5 |   -0.480 |  0.009 | --
-- |  6 |   0.525 |  0.137 | --
-- |  7 |   1.530 |  0.084 | --
-- |  8 |   2.535 |  0.050 | --
-- |  9 |   3.540 |  0.032 | --
-- | 10 |   4.545 |  0.022 | --
-- | 11 |   5.550 |  0.016 | --
-----

```

Lisa 15: Lisaülesanne 2. eelseadistatud väärtuste komplektiga

```

frank@frankpc:~/Documents/C Programming/Homework01/Final$ ./HW1_221925IVCM_Fra
nk_Christopher_Kirch --TEST3

The program solves function  $y = (2x+1) / \sqrt{(2x^2 + 3x + 4)^3 - 7}$ 
The function's domain is:  $[-\infty; \infty]$ .

-----
-- Running TESTSET 3! --
-----
-- min    -4.500      --
-- max     5.950      --
-- step     11        --
-----

----- RESULTS -----
-----
-- | ind |      X |      Y | --
-----
-- |  1 | -4.500 | -0.046 | --
-- |  2 | -3.455 | -0.081 | --
-- |  3 | -2.410 | -0.158 | --
-- |  4 | -1.365 | -0.271 | --
-- |  5 | -0.320 |  0.069 | --
-- |  6 |  0.725 |  0.127 | --
-- |  7 |  1.770 |  0.074 | --
-- |  8 |  2.815 |  0.044 | --
-- |  9 |  3.860 |  0.029 | --
-- | 10 |  4.905 |  0.020 | --
-- | 11 |  5.950 |  0.014 | --
-----

```

Lisa 16: Lisaülesanne 3. eelseadistatud väärtuste komplektiga